

Curación resuelta

Los 41 módulos decididos, y los nombres que no llegaban a módulo.

Cómo leerlo

Una fila por módulo, en el mismo orden que la hoja pendiente, para poder cotejarlas línea a línea. La marca elegida va recuadrada; las otras cuatro quedan en gris.

+	nombre que la tabla de citas del módulo no ofrecía. Sale de la declaración, no de la ruta. Son diez, y todos son el tipo base al que pertenecen los identificadores que sí venían.
†	padre que no está entre los candidatos mostrados. Las listas por área vienen cortadas a diez en orden alfabético, así que en algebra, topology, analysis, geometry y number-theory el candidato puede existir más allá del corte; en computation, set-theory, probability y logic la lista mostrada es completa y el dagger significa crearlo.
nombres	se queda todo el que <i>sea</i> la noción, incluidas las dos formas cuando Mathlib da tipo y predicado. Se van operaciones, constructores, coerciones, empaquetados y auxiliares de demostración.

Lo que sale

Marcas: **C** 8 · **S** 7 · **F** 15 · **O** 2 · **T** 9. Entran 32 nodos, salen 9 módulos.

De los 150 identificadores que ofrece la hoja se quedan 30; los otros 10 nombres del recuento hay que leerlos de su declaración porque el orden por citas no los alcanza.

Cuatro aristas caen entre nodos nuevos, no hacia el área: `Class→ZFSet→Lists`, `Similar→Congruent`, `SlashInvariantForm→ModularForm`, `Derivation→LeftInvariantDerivation`. El padre correcto era otro módulo de esta misma tanda, no un nodo viejo.

Y tres huecos que los módulos destapan. El área set-theory no tiene raíz: cardinal-arithmetic, forcing, large-cardinals y descriptive-set-theory son las cuatro especializaciones de *conjunto*, y ninguna puede ser su padre; `Class` entra como raíz. No hay nodo de ordinales, y por eso `SetTheory.Ordinal.Notation` se va sin quedar cubierto por nada. No hay nodo de formas modulares —entre modular-arithmetic y number-fields la lista del área es contigua, así que no está cortado, falta—, y lo abre `SlashInvariantForm`.

Computability área computation

Computability.AkraBazzi.SumTransform

C **S** **F** **O** **T**

por qué El contenido es el teorema de Akra–Bazzi, y `asymptBound`, `sumTransform`, `min_bi` y `max_bi` son piezas de su demostración. Teorema, no objeto: el mismo caso que `prime-factorization`.

Computability.DFA

C **S** **F** **O** **T**

padre `turing-machines`

se quedan `DFA+`, `Language.IsRegular`

por qué Los autómatas son objetos y `comap` y `reindex` son el reíndice funtorial entre ellos. En la jerarquía de modelos de máquina, el AFD es el caso sin escritura ni movimiento libre. `Language.IsRegular` se declara aquí y nombra justo lo que este nodo decide.

Computability.Language

C **S** **F** **O** **T**

padre `computability-theory`

se quedan `Language+`

por qué Un lenguaje es un conjunto de palabras: objetos, ordenados por inclusión, con `map` funtorial en el alfabeto. `reverse` y `map` son operaciones; `Symbol` es el alfabeto de las gramáticas, otro concepto.

Computability.Partrec

C **S** **F** **O** **T**

padre `recursion-theory`

se quedan `Partrec`, `Computable`

por qué No son una colección de objetos: son la clase de flechas que se deja calcular, cerrada por composición. `Nat.Partrec` es la definición primitiva sobre los naturales y `rfind` el operador de búsqueda con el que se construye.

Computability.Primrec.BasicCS **F** OT

padre recursion-theory

se quedan Primrec

por qué La misma clase de flechas, más estrecha. Primcodable es la codificación que fija los objetos del ambiente, no la noción, y Nat.Primrec su caso sobre los naturales.

Control sin área en el grafo**Control.Bitraversable.Basic Control.Fix Control.Functor.Multivariate Control.Monad.Cont** CSFO **T**

por qué Es la interfaz de efectos de Lean, y sí, Bitraversable, MvFunctor y ContT son funtores: endofuntores sobre Type. No unen dos vértices, y un lazo no es una dependencia. Part.fix es el punto fijo con el que Lean define funciones parciales, maquinaria de definición. La rama entera no toca.

RingTheory área algebra**RingTheory.DedekindDomain.Different**CS **F** OT

padre commutative-algebra

se quedan differentIdeal

por qué El diferente no es una colección que se recorra: es lo que la traza produce a partir de una extensión de dominios de Dedekind. FractionalIdeal.dual y Submodule.traceDual son el dual respecto de la forma traza con el que se construye.

RingTheory.Derivation.BasicCS **F** OT

padre commutative-algebra

se quedan Derivation

por qué Una derivación es una flecha con Leibniz, y el tipo nombra la clase entera. restrictScalars, llcomp y coeFnAddMonoidHom son operaciones y empaquetado.

RingTheory.GradedAlgebra.Homogeneous.IdealCS **S** FOT

padre commutative-algebra

se quedan HomogeneousIdeal, Ideal.IsHomogeneous

por qué Ser homogéneo es una propiedad sobre los ideales: subcategoría plena, y la inclusión es la arista. Las dos formas nombran la misma noción y los enunciados usan ambas. homogeneousHull es una operación y irrelevant un ideal concreto.

RingTheory.GradedAlgebra.HomogeneousLocalizationCS **F** OT

padre commutative-algebra

se quedan HomogeneousLocalization

por qué Localizar es una construcción, no una clase de objetos: va de álgebras graduadas a anillos. NumDenSameDeg es su presentación por fracciones del mismo grado y awayMap su funtorialidad.

SetTheory área set-theory**SetTheory.Lists**CS **S** FOT

padre ZFSet

se quedan Lists⁺

por qué Modelo computable de los conjuntos hereditariamente finitos: la subcategoría plena de los de rango y ramificación finitos. Lists' es la mitad de la definición mutua y toList/ofList la codificación.

SetTheory.Ordinal.NotationCSFO **T**

por qué ONote y NONote son la forma normal de Cantor como dato computable, y repr la denotación hacia los ordinales de verdad. Notación, no objeto. Lo que falta aquí no es este módulo, es un nodo de ordinales.

SetTheory.ZFC.BasicCS **S** FOT

padre Class

se quedan ZFSet⁺

por qué Un conjunto es una clase pequeña, y Class.ofSet es literalmente la inclusión que la marca S pide. Nonempty, range, prod e image son operaciones del universo, no el universo.

SetTheory.ZFC.Class

CSFOT

padre — raíz del área
 se quedan Class
 por qué Class = Set ZFSet, y de ahí cuelga todo lo demás. Es la raíz que el área no tiene: sus cuatro nodos son especializaciones suyas, así que ninguno puede ser su padre. univ y powerset son operaciones, ofSet la arista de la fila anterior.

Topology área topology**Topology.Category.TopCat.Basic**

CSFOT

padre point-set-topology
 se quedan TopCat
 por qué Es la categoría misma, empaquetada para poder tomar límites en ella. ofHom y Hom.hom son el ida y vuelta con las funciones continuas, y trivial una topología concreta.

Topology.MetricSpace.Congruence

CSFOT

padre Similar
 se quedan Congruent
 por qué Congruente es que exista una correspondencia que conserve todas las distancias: clase de flechas. Es el caso de razón 1 de Similar, y por eso cuelga de él y no del área.

Topology.MetricSpace.Holder

CSFOT

padre point-set-topology
 se quedan HolderWith, HolderOnWith
 por qué Condición de crecimiento sobre una aplicación; especializa la continuidad. La global y la relativa a un conjunto son la misma noción y las dos aparecen en enunciados.

Topology.MetricSpace.Similarity

CSFOT

padre euclidean-geometry
 se quedan Similar
 por qué Lo mismo con razón arbitraria. El módulo está en topología pero el concepto es euclidiano: manda el padre, no la carpeta.

Algebra área algebra**Algebra.Lie.Basic**

CSFOT

padre lie-algebras[†]
 se quedan LieHom⁺, LieModuleHom⁺
 por qué Los cuatro identificadores más citados son .comp y .symm: operaciones sobre flechas, no las flechas. Lo que se queda son los tipos de morfismo. El padre cae más allá del corte alfabético; si no existe hay que crearlo, porque Algebra.Lie es una rama entera de Mathlib sin nodo.

Algebra.Module.CharacterModule

CSFOT

padre module-theory
 se quedan CharacterModule⁺
 por qué Es el funtor contravariante que manda un módulo a sus homomorfismos aditivos en $\mathbb{Q}/\mathbb{Z}$, no una clase de objetos. uncurry y homEquiv son sus adjunciones, dual su iteración.

Algebra.Module.Injective

CSFOT

padre module-theory
 se quedan Module.Injective
 por qué Propiedad sobre módulos: subcategoría plena. Module.Baer es el criterio equivalente, un teorema con forma de definición, y ExtensionOf/extensionOfMax la maquinaria de Zorn que lo prueba.

NumberTheory área number-theory**NumberTheory.ModularForms.Basic**

CSFOT

padre SlashInvariantForm
 se quedan ModularForm
 por qué ModularForm extiende SlashInvariantForm con holomorfía y acotación en las cúspides: el extends de Mathlib es la inclusión plena. ModularFormClass es la interfaz de coerción.

NumberTheory.ModularForms.SlashInvariantForms**C** **S** **F** **O** **T**

padre analytic-number-theory

se quedan SlashInvariantForm

por qué La invariancia bajo la acción slash de peso k es la que define los objetos; la holomorfía llega después. prod y const son la estructura de anillo graduado sobre ellos.**NumberTheory.NumberField.InfinitePlace.Basic****C** **S** **F** **O** **T**

padre number-fields

se quedan NumberField.InfinitePlace

por qué Una plaza infinita es un valor absoluto arquimediano sobre el cuerpo, es decir una flecha hacia $\mathbb{R}$ módulo equivalencia; el tipo es la clase de esas flechas. IsReal, IsComplex y embedding la parten en casos.**AlgebraicTopology** área topology**AlgebraicTopology.SimplexCategory.Defs****C** **S** **F** **O** **T**

padre algebraic-topology

se quedan SimplexCategory

por qué Categoría en sentido literal: es el índice de todo objeto simplicial. Hom.comp es su composición y Truncated.inclusion la inclusión de la truncada.

AlgebraicTopology.SimplexCategory.GeneratorsRelations.Basic**C** **S** **F** **O** **T**

por qué SimplexCategoryGenRel es SimplexCategory presentada por caras, degeneraciones y relaciones simpliciales: la misma categoría con otro nombre, y FreeSimplexQuiver el carcaj del que se cocienta. Dos nombres para un objeto gastan dos plazas y no añaden concepto.

Analysis área analysis**Analysis.Complex.Circle****C** **S** **F** **O** **T**

padre circle-geometry

se quedan Circle

por qué Un objeto, el círculo unidad del plano complejo con su estructura de grupo: vértice degenerado. exp, toUnits y coeHom son las flechas que entran y salen de él.

Analysis.Complex.UpperHalfPlane.Basic**C** **S** **F** **O** **T**

padre complex-analysis

se quedan UpperHalfPlane

por qué Otro objeto individual. re, im son coordenadas e I un punto suyo. Es el dominio sobre el que viven las formas modulares de la rama anterior.

Data sin área en el grafo**Data.PFun****C** **S** **F** **O** **T**

padre computability-theory

se quedan PFun^+

por qué Una función parcial es una flecha, y PFun la clase entera; sobre ella se define Partrec. La rama no es área, pero este concepto sí tiene dónde colgar. comp, restrict, lift e image son operaciones.

Data.TypeVec**C** **S** **F** **O** **T**

por qué Vectores de tipos para construir tipos inductivos multivariados. Infraestructura de Lean, como la rama Control, y por el mismo motivo.

Dynamics área probability**Dynamics.Circle.RotationNumber.TranslationNumber****C** **S** **F** **O** **T**

padre ergodic-theory

se quedan CircleDeg1Lift, CircleDeg1Lift.translationNumber⁺

por qué Los levantamientos de grado uno son una clase de flechas de la recta en sí misma, y el número de traslación es el invariante que el módulo existe para definir. La tabla no lo ofrece —ofrece transnumAuxSeq, la sucesión auxiliar con la que se prueba que el límite existe—, así que hay que leerlo de su declaración.

Dynamics.Flow**C** **S** **F** **O** **T**

padre ergodic-theory
 se quedan Flow^+
 por qué Un flujo es el sistema dinámico entero, objeto y no flecha: el espacio más la acción continua. orbit, restrict y reverse son operaciones sobre él, e IsInvariant una propiedad de subconjuntos.

Geometry área geometry**Geometry.Manifold.Algebra.LeftInvariantDerivation****C** **S** **F** **O** **T**

padre Derivation
 se quedan LeftInvariantDerivation
 por qué Las derivaciones invariantes por traslación izquierda son una subclase de las derivaciones, así que el padre es la fila de RingTheory.Derivation.Basic y no el área. evalAt es la evaluación en un punto, que es la que da el isomorfismo con el espacio tangente en la identidad.

Geometry.Manifold.Algebra.LieGroup**C** **S** **F** **O** **T**

padre differential-geometry
 se quedan LieGroup
 por qué Objetos con dos estructuras compatibles. No es subcategoría plena de las variedades, porque los morfismos son los homomorfismos suaves y no todas las aplicaciones suaves: por eso C y no S. LieAddGroup es el duplicado aditivo y ContMDiffInv0 una hipótesis técnica.

AlgebraicGeometry área geometry**AlgebraicGeometry.ProjectiveSpectrum.Topology****C** **S** **F** **O** **T**

padre algebraic-geometry
 se quedan ProjectiveSpectrum
 por qué Es la construcción que lleva un anillo graduado a un espacio, igual que Spec. zeroLocus y vanishingIdeal son la conexión de Galois que la define y basicOpen la base de la topología.

LinearAlgebra área algebra**LinearAlgebra.AffineSpace.FiniteDimensional****C** **S** **F** **O** **T**

padre euclidean-geometry
 se quedan Collinear, Coplanar
 por qué La misma noción con el rango afín acotado por 1 y por 2: subcategoría plena de los subconjuntos de un espacio afín. Van juntas en un nodo, porque separarlas duplicaría la arista sin ganar nada.

ModelTheory área logic**ModelTheory.Arithmetic.Presburger.Semilinear.Defs****C** **S** **F** **O** **T**

padre model-theory
 se quedan IsSemilinearSet, IsLinearSet
 por qué Los semilineales son exactamente los definibles en la aritmética de Presburger: una propiedad sobre subconjuntos de $\mathbb{N}^n$. Los IsProper... son la variante normalizada que usa la demostración.

Order sin área en el grafo

Order.PFilterCSFO **T**

por qué Un PFilter es un filtro sobre un preorden. Order no es área del grafo y un nodo solo no la justifica: sin padre la arista no existiría y el nombre quedaría suelto. Si alguna vez entra la teoría de órdenes, este es el primer candidato.

Los nombres que no llegan a módulo

Son ocho etiquetas y nueve nombres: turing-machines gasta dos plazas en la misma etiqueta.

etiqueta	decisión	qué escribir
cic	quitar el nombre	Mathlib no declara el cálculo de construcciones inductivas. CategoryTheory.types sí existe —es la instancia de categoría sobre Type u, en Mathlib.CategoryTheory.Types.Basic— pero nombra otra cosa. Si la etiqueta era la categoría de tipos y no el cálculo, entonces la decisión es la de abajo y el módulo es ese.
conditional-expectation	asignar el módulo a mano	Mathlib.MeasureTheory.Function.ConditionalExpectation.Basic
lebesgue-integration	asignar el módulo a mano	Mathlib.MeasureTheory.Integral.Bochner.Basic. Es la integral de Bochner; la de Lebesgue a valores en los reales extendidos no negativos es MeasureTheory.lintegral, en Mathlib.MeasureTheory.Integral.Lebesgue.Basic. Los enunciados usan la primera.
module-theory	el nombre está mal	ModuleCat. ModuleCat R es una aplicación: el anillo es argumento, no parte del nombre.
projective-varieties	asignar el módulo a mano	Mathlib.AlgebraicGeometry.ProjectiveSpectrum.Scheme. El nombre es correcto y no hay mejor: Mathlib no tiene «variedad proyectiva», sólo el Proj de un álgebra graduada.
quotient-groups	el nombre está mal	QuotientGroup.mk', la proyección canónica al cociente. QuotientGroup es un espacio de nombres y con marca F lo que hace falta es la flecha. Si se quisiera el objeto, sería HasQuotient.Quotient.
turing-machines	el nombre está mal	Turing.TM0.Machine, en Mathlib.Computability.PostTuringMachine desde que se partió TuringMachine.lean.
turing-machines	quitar el nombre	Es la segunda plaza de la misma etiqueta. Y TM1 no tiene una declaración que sea «la máquina»: el modelo se da como familia indexada de Turing.TM1.Stmt. Si se quiere TM1 explícito, ese es el nombre.
sheafed-space-complexes	el nombre está mal	AlgebraicGeometry.SheafedSpace. Lo que había era una aplicación con el argumento fijado. Si la etiqueta quiere justamente los espacios anillados en complejos de cocadenas, la abreviatura hay que declararla en el proyecto: Mathlib no la trae.

Lo que esta tanda cuesta

32 nodos y 40 nombres nuevos, contra 150 identificadores ofrecidos. La regla que más recorta es la tercera: de cada módulo sobrevive el tipo que nombra la noción y casi nada más, porque .comp, .symm y .restrict son el mismo puñado de operaciones repetidas en cuarenta ramas: ofrecerlas cuarenta veces no distingue nada.

Diez de esos cuarenta nombres no venían en la hoja, y no es casualidad. La columna de citas cuenta el último componente del nombre, no la declaración: .comp vale 1617 en LieHom, LieModuleHom, SimplexCategory.Hom, PFun y TypeVec a la vez; .prod vale 1046 en cuatro módulos sin relación; .symm 2247 en dos; .restrict 702, .image 415, .ofNat 409, .const 322, .reverse 164, .repr 159, .toList 130, .dual 110, .embedding 53, .coeFnAddMonoidHom 13, todos repetidos exactos en módulos distintos. Un sufijo genérico se lleva la cuenta de todo Mathlib y por eso siempre gana al tipo que lo declara, que se llama de una sola forma y en un solo sitio. Con ese orden, los cuatro primeros de un módulo son casi siempre operaciones, y el nombre que hace falta no aparece. Contar por nombre completo lo arregla.